

Deep Learning Homework 1

José Machado, Gil Jorge, Elias Heimda

December 12, 2025

Group Contributions

For Question 1, Elias Heimdal was responsible for the implementation of the code, generation of all plots, and the writing of the corresponding report section, summarizing all the contents related to Question 1.

For Question 2, José Machado was responsible for the implementation of the code, generation of all plots, and the writing of the corresponding report section, summarizing all the contents related to Question 2.

For Question 3, Gil Jorge was responsible for the entirety of the content and writing of the corresponding report section, summarizing all the contents related to Question 3.

After everyone completed their respective question, we cross checked each others work.

Question 1: Handwritten Letter Classification (35 points)

1. Perceptron (5 points)

We implemented a multi-class perceptron to classify the EMNIST handwritten letters. The model was trained for 20 epochs, and we implemented a checkpointing mechanism to save the model weights whenever the validation accuracy improved over previous epochs. The train and validity accuracies for the epochs goes as follows:

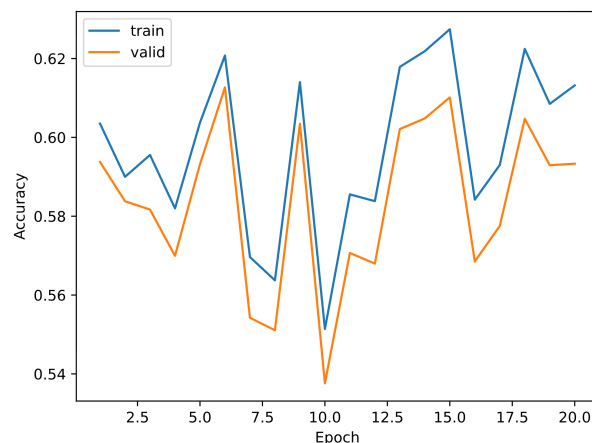


Figure 1: Perceptron training and validation accuracy over 20 epochs.

The model steadily learns the patterns in the data, though as a linear classifier, its capacity is limited, resulting in a best-performing checkpoint achieving a **test accuracy of 0.6111**.

The experiment can be reproduced using the following command:

```
python hw1-perceptron.py --save-path model.pkl
```

2. Logistic Regression (20 points)

(a) Implementation with ℓ_2 regularization (10 points) We implemented a logistic regression classifier trained via Stochastic Gradient Descent (SGD) with a batch size of 1. To improve generalization, we incorporated ℓ_2 regularization into the weight update rule.

For this experiment, we used a learning rate of 0.0001 and an ℓ_2 penalty of 0.00001.

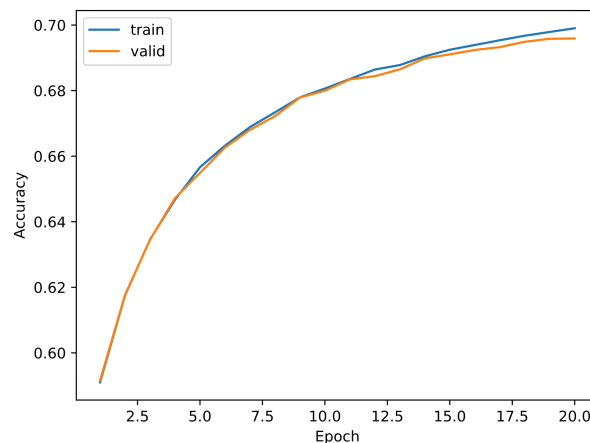


Figure 2: Logistic Regression training and validation accuracy.

Figure 2 shows the training trajectory over 20 epochs, showing it outperformed the simple perceptron, achieving a best **test accuracy of 0.6975** (recorded at epoch 20).

Command to run the experiment:

```
python hw1-logreg.py \
--save-path best_logreg.pkl \
--learning-rate 0.0001 \
--l2-penalty 0.00001 \
--epochs 20
```

(b) Alternative Feature Representation (5 points) **Proposed Feature:** We propose using **downsampled images** as an alternative feature representation to improve training efficiency while maintaining structural information. Instead of using the full 784-dimensional flattened pixel vector (28×28), we apply 2×2 average pooling to reduce the image to $14 \times 14 = 196$ features.

Average pooling divides the image into non-overlapping 2×2 blocks and replaces each block with its mean value. Given an input image $I \in \mathbb{R}^{28 \times 28}$, the pooled output

$P \in \mathbb{R}^{14 \times 14}$ is computed as:

$$P_{i,j} = \frac{1}{4} \sum_{m=0}^1 \sum_{n=0}^1 I_{2i+m, 2j+n}, \quad i, j \in \{0, 1, \dots, 13\} \quad (1)$$

The implementation uses vectorized NumPy operations for efficiency:

```
def extract_downsampled_features(X, original_size=28, pool_size=2):
    n_samples = X.shape[0]
    new_size = original_size // pool_size # 14
    imgs = X.reshape(n_samples, original_size, original_size)
    imgs = imgs.reshape(n_samples, new_size, pool_size, new_size, pool_size)
    pooled = imgs.mean(axis=(2, 4)) # Average over 2x2 blocks
    return pooled.reshape(n_samples, -1)
```

This is an effective feature representation as it exploits spatial redundancy. Neighboring pixels in images are highly correlated, so by averaging 2×2 blocks we capture the essential structural information while reducing noise.

The primary advantage of this method is **efficiency**. With 196 features instead of 784, each SGD update requires $\sim 4\times$ fewer multiply-accumulate operations, making training substantially faster—a valuable trait for extensive hyperparameter grid searches. Similar approaches have been utilized by Cohen et al. [1] on the EMNIST dataset and by LeCun et al. [2] for subsampling in convolutional networks.

We compared three feature representations using logistic regression with identical hyperparameters (learning rate = 10^{-4} , ℓ_2 penalty = 10^{-5} , 20 epochs):

Feature Representation	Dimensions	Val. Acc.	Test Acc.	Time (s)
Raw Pixels	785	69.6%	69.7%	134
Downsampled (14×14)	197	65.7%	65.3%	48
HOG	145	61.3%	61.7%	41

Table 1: Comparison of feature representations for logistic regression on EMNIST Letters.

The downsampled representation achieves **93.7% of the raw pixel accuracy** (65.3% vs 69.7%) while training **2.8 $\times$ faster**. This represents a favorable accuracy-efficiency tradeoff. We also tested HOG (Histogram of Oriented Gradients) features [3], which capture edge orientations. However, HOG performed notably worse (61.7% test accuracy), likely because it discards spatial information relevant for distinguishing visually similar letters (e.g., “O” vs “Q”, “E” vs “F”).

Command to run:

```
python hw1-logreg-b-downsample.py --save-path logreg-downsample.pkl
```



(c) **Grid Search for Hyperparameters (5 points)** To find the optimal model configuration, we performed a grid search over 12 combinations: 3 learning rates $\times$ 2 ℓ_2 penalties $\times$ 2 feature representations. Each configuration was trained for 20 epochs, and the checkpoint with the best validation accuracy was retained.

The validation accuracies for all explored configurations are sorted below:

Learning Rate	ℓ_2 Penalty	Features	Validation Accuracy
0.001	10^{-5}	pixels	0.7184
0.001	10^{-4}	pixels	0.7170
0.0005	10^{-5}	pixels	0.7165
0.0005	10^{-4}	pixels	0.7142
0.001	10^{-5}	downsampled	0.7082
0.001	10^{-4}	downsampled	0.7049
0.0005	10^{-5}	downsampled	0.6998
0.0005	10^{-4}	downsampled	0.6978
0.0001	10^{-4}	pixels	0.6960
0.0001	10^{-5}	pixels	0.6959
0.0001	10^{-5}	downsampled	0.6569
0.0001	10^{-4}	downsampled	0.6566

Table 2: Validation accuracy for all 12 grid search configurations, sorted by performance.

The best-performing configuration was:

- Learning rate: 0.001
- ℓ_2 penalty: 10^{-5}
- Feature representation: original pixels (784 dimensions)

Using this optimal configuration, we achieved a final **test accuracy of 0.7208**.

Command to run the grid search:

```
python hw1-logreg-gridsearch.py --data-path emnist-letters.npz
```

3. Multi-Layer Perceptron (10 points)

Moving beyond linear classifiers, we implemented a Multi-Layer Perceptron (MLP) with a single hidden layer. The implementation was done from scratch, including the manual derivation and coding of the backpropagation algorithm for gradient computation.

The network architecture consists of:

- **Input layer:** 784 features (flattened 28×28 pixel images)
- **Hidden layer:** 100 units with ReLU activation
- **Output layer:** 26 units with softmax activation (minimizing cross-entropy loss)

Training Configuration

The training hyperparameters and initialization schemes are detailed in Table 3.

Hyperparameter	Value
Hidden units	100
Activation function	ReLU
Learning rate	0.001
Batch size	1 (SGD)
Epochs	20
Weight initialization	$w_{ij} \sim \mathcal{N}(\mu = 0.1, \sigma^2 = 0.01)$
Bias initialization	Zero vectors

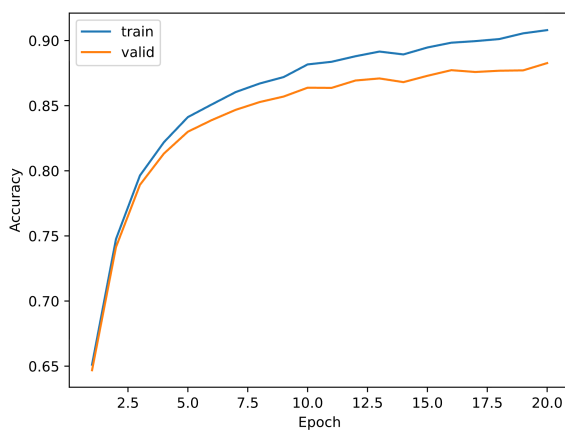
Table 3: MLP training hyperparameters.

Results

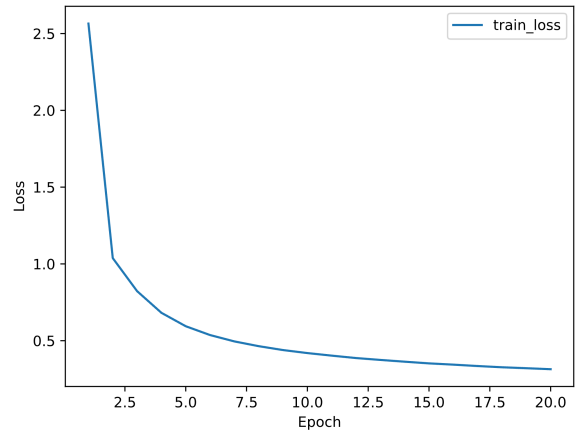
The MLP significantly outperformed the linear classifiers, demonstrating the value of the non-linear hidden layer. Figure 3 displays the training metrics, showing a smooth decrease in loss and a steady increase in accuracy.

Metric	Value
Best validation accuracy	0.8827 (epoch 20)
Test accuracy	0.8799
Final training accuracy	0.9080
Final training loss	0.3144
Training time	7 min 14 sec

Table 4: MLP performance on EMNIST Letters.



(a) Training and validation accuracy



(b) Training loss

Figure 3: MLP training curves over 20 epochs.

Command to run the MLP training:

```
python hw1-mlp.py --data-path emnist-letters.npz --save-path mlp_best.pkl
```

Question 2: Feedforward Neural Networks (35 points)

1. Implementation Warmup (5 points)

The class `FeedforwardNetwork(nn.Module)` was implemented by constructing an internal module `self.fnn`, defined as an `nn.Sequential` container. This sequential module encodes the exact series of transformations that the feedforward neural network applies during the forward pass. Because of this design, the `forward` method reduces to a simple and clean implementation, `return self.fnn(x)`

In addition to the network class, several helper functions were implemented. Each function was written to strictly follow the argument types and return types specified in the docstrings of the skeleton code.

2. Towards an Infinite-Width One-Layer FFN (18 points)

(a) Grid Search Over Widths (10 points)

To study how width affects performance, we trained a one-hidden-layer feedforward neural network for five different widths: $\{16, 32, 64, 128, 256\}$. For each width, we performed a grid search over 80 hyperparameter combinations, obtained from:

- Four learning rates: $\{0.09, 0.12, 0.15, 0.17\}$,
- Two dropout values: $\{0.0, 0.1\}$,
- Two L2 weight decay values: $\{0.0, 10^{-4}\}$,

The learning rate values are such because grid search with smaller values always resulted in the best performing model to be the one with highest learning rate, indicating that the learning rates were too small. After 0.15 the model tended to get worse results, so the final considered grid search considers learning rates around 0.15. The dropout and l2 values were also mildly tuned so the best performing models don't all have 0 on both these hyperparameters.

For all combinations, we used SGD as the optimizer, ReLU activation, batch size 64, and trained for 30 epochs. For each configuration, we recorded the best validation accuracy achieved during training. All results for the full 80-configuration grid search are available in the file `Q2_outputs/all_configs.csv`, and the script used to run this experiment is `hw1_ffn_q2_a.py`.

The following table summarizes the best-performing configuration for each width. These values are available in `Q2_outputs/best_per_width.csv`.

Width	Learning Rate	Dropout	L2	Best Val. Acc
16	0.12	0.0	0.0	0.7852
32	0.15	0.0	10^{-4}	0.8502
64	0.12	0.0	10^{-4}	0.8812
128	0.15	0.1	10^{-4}	0.9015
256	0.15	0.1	10^{-4}	0.9128

Table 5: Best-performing configuration for each width.

Observation: We can briefly observe that the best validation accuracy increases with the width of the hidden layer. This indicates that wider single layer FFNs are more expressive, achieving better accuracy on the validation data set.

(b) Best Overall Model (4 points) Among all widths and hyperparameter combinations explored, the best validation accuracy was achieved by the network with:

Width	LR	Dropout	L2
256	0.15	0.1	10^{-4}

Table 6: grid search hyperparameters of the model with best validation accuracy on grid search

Using the code present in `hw1_ffn_q2_b.py`, we trained a model with hyperparameters of Table 6 over 30 epochs, registering the following results for training loss and validity accuracy, respectively:

The loss decreases smoothly and levels off by the final epochs, indicating convergence. The validation accuracy similarly plateaus without signs of overfitting or underfitting, suggesting stable training behavior.

The test accuracy of the model with best validation accuracy is **0.9095**, which is consistent with the validation performance with accuracy of **0.9128**, which indicates good generalization to unseen data.

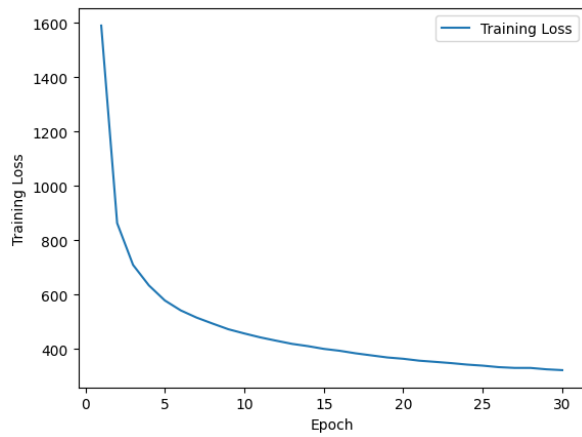


Figure 4: Training loss over epochs for the best-performing model.

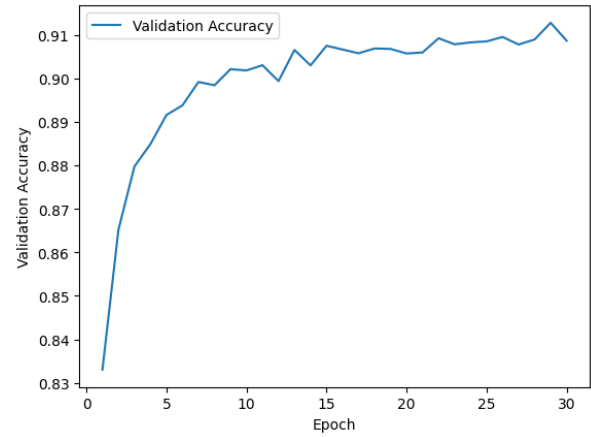


Figure 5: Validation accuracy over epochs for the best-performing model.

(c) Effect of Width on Training Accuracy (4 points) To analyze how the width of the hidden layer affects the network's capacity to interpolate the training data, we will analyze the final training accuracy as function of hidden-layer width.

Using the code present in `hw1_ffn_q3_c.py`, the best-performing model for each hidden-layer width explored in the grid search are going to be retrained under exact same circumstances (hyperparameters, activation, optimizer, batch size), over 30 epochs, so that we can obtain the final training accuracy for each model. The following plot was then produced, which can be found in `Q3_outputs/training_acc_vs.width.png`:

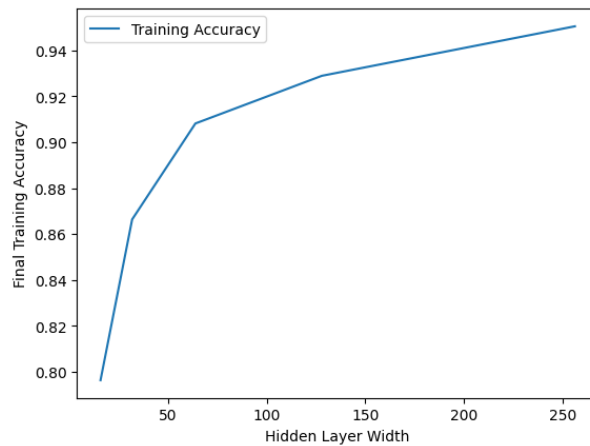


Figure 6: Final training accuracy as a function of hidden layer width. Wider networks achieve higher training accuracy, reflecting increased capacity to fit the training data.

A clear trend can be observed where the training accuracy increases consistently with the width of the hidden layer. This indicates that wider networks have greater representational capacity, allowing them to interpolate the training data more effectively within the same number of training epochs.

This supports the Universal Approximation Theorem, which states that a feedforward neural network with a single hidden layer and sufficiently many neurons can approximate any continuous function on a compact domain to arbitrary accuracy, meaning that as we increase the width of a one-layer FFN, the network's capacity to fit the training data increases. The expected theoretical training accuracy for an infinite-width one-layer FFN is then 100%, which neglects computational imprecisions.

3. Effect of Depth in Vanilla FFNs (12 points)

(a) Grid Search Over Depth (4 points) To study the effect of Depth in Vanilla FFNs, models with $L \in \{1, 3, 5, 7, 9\}$ are trained for 30 epochs with batch size of 64. The regarding hyperparameters correspond to the hyperparameters of the best-performing model with 32 width in Question 2.2.a), present in the following table:

LR	Dropout	L2	Activation	Optimizer
0.15	0.0	10^{-4}	ReLU	SGD

Table 7: grid hyperparameter configuration of the best-performing 32-unit model for 30 epochs in question 2.2.a).

Using the code in `hw1_ffn_q3_a.py`, the models were trained and the highest validation accuracy was obtained for each model:

Depth (L)	Best Validation Accuracy
1	0.8492
3	0.8671
5	0.8659
7	0.8564
9	0.8385

Table 8: Best validation accuracy per depth for 32-unit FFNs.

This table shows that for $L = 1 - 3$ the validation accuracy increases, being $L = 3$ the model with highest registered validation accuracy. For $L > 3$, there is a tendency for the validation accuracy to decrease.

(b) Best Depth Model (4 points) From Table 8, the model with $L = 3$ obtained the best validation accuracy for 30 epochs. The code in `hw1_ffn_q3_b.py` trains this model for 30 epochs with the same hyper-parameters while recording training loss and the validation accuracy per epoch, which were then plotted in the following figures, respectively:

The training loss decreases steadily and stabilizes by the end of the training, indicating convergence within 30 epochs. The validation accuracy plateaus with no significant drop, showing no signs of significant overfitting.

The test accuracy was found to be **0.8630** for the epoch with the best validation accuracy (epoch 26), which is very similar to the best validation accuracy **0.8671**, confirming good generalization.

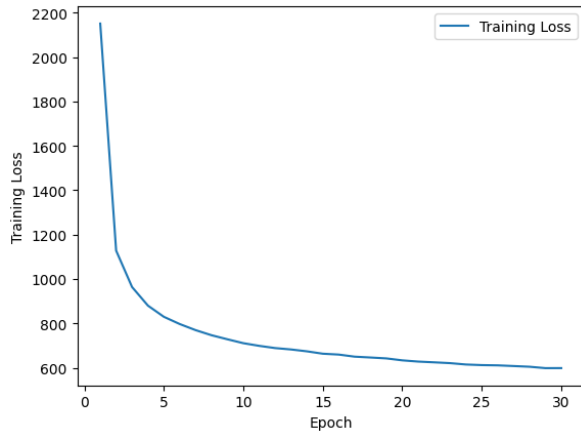


Figure 7: Training loss over epochs for the best-performing model.

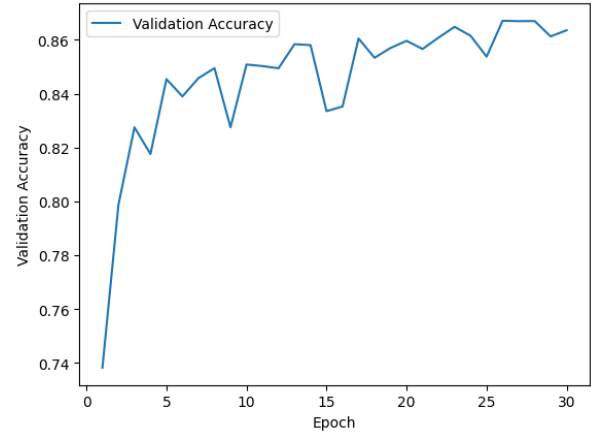


Figure 8: Validation accuracy over epochs for the best-performing model.

(c) Training Accuracy vs Depth (4 points) To see how depth affects the model’s ability to fit the training data, we plot in the following figure the training accuracy of the last epoch for all of the models used $L \in \{1, 3, 5, 7, 9\}$.

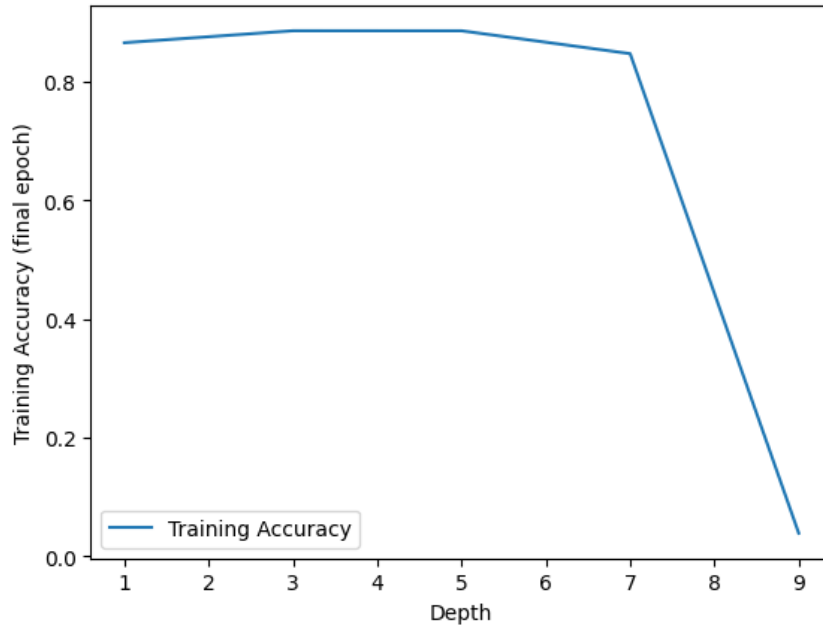


Figure 9: Final training accuracy as a function of depth for 32-unit per layer FFNs.

This figure shows that while training accuracy improves initially up to $L = 3$, it declines for deeper networks ($L > 3$), having an extreme decrease for $L = 9$. Theoretically, deeper architectures possess a higher representational capacity, which should allow perfect interpolation of the training data. However, the observed decrease in training accuracy suggests that the models are facing optimization difficulties that limit the models capacity

to fit the data. As depth increases the network becomes harder to train due to the vanishing or exploding gradient problem, which prevents efficient weight updates in earlier layers. Consequently, despite their potential to fit the data, the deeper models fail to converge to an optimal solution within the selected number of epochs, illustrating that increased depth can hinder the model's ability to learn for a certain limit of epochs.

Question 3

3.1

Notice $\|y - z\|^2 = \sum_{i=1}^K (y_i - z_i)^2$. To minimize this we minimize each term, $\|y_i - z_i\| = |y_i - z_i|$. With the only constraint that $y_i \geq 0$, when $z \leq 0$, $\arg \min \|y - z\|$ is $y = 0$, whereas if $z \geq 0$, y can be equal to z which minimizes the distance between them (0). Thus we obtain the result, assuming $z \geq 0$ means $z_i \geq 0$, for $i \in \{1, \dots, K\}$.

$$y_i = \max(0, z_i) = \text{relu}(z_i)$$

3.2

Softmax Invariance

$$\text{softmax}(z + c)_i = \frac{e^{z_i + c}}{\sum_j e^{z_j + c}} = \frac{e^{z_i} e^c}{e^c \sum_j e^{z_j}} = \text{softmax}(z)_i$$

The constant e^c cancels out.

Sparsemax Invariance

$\text{sparsemax}(z) = \text{relu}(z - \tau)$. Since τ has to be such as to satisfy the normalization constraint, writing $\text{relu}(z_i - \tau')$ as $\max(0, z_i - \tau')$, we have to have that $\sum_i \max(0, z_i + c - \tau') = 1$. Notice $\tau' = \tau + c$ gives us the normalization constraint for z , and since τ is the unique scalar that satisfies this condition, $\tau' = \tau + c$, so:

$$\max(0, z_i + c - \tau') = \max(0, z_i + c - (\tau + c)) = \max(0, z_i - \tau)$$

Thus getting the result.

Relumax Invariance

Let the equation for the i -th component of relumax be (1):

$$\text{relumax}_b(z)_i = \frac{\text{relu}(z_i - \max(z) + b)}{\sum_j \text{relu}(z_j - \max(z) + b)} \quad (1)$$

$\max(z + c) = \max(z) + c$ because c is constant, so (1) turns into:

$$\frac{\text{relu}(z_i + c - (\max(z) + c) + b)}{\sum_j \text{relu}(z_j + c - (\max(z) + c) + b)}$$

Expanding the terms inside the ReLU:

$$= \frac{\text{relu}(z_i + c - \max(z) - c + b)}{\sum_j \text{relu}(z_j + c - \max(z) - c + b)} = \frac{\text{relu}(z_i - \max(z) + b)}{\sum_j \text{relu}(z_j - \max(z) + b)}$$

Which cancels to get $\text{relumax}_b(z)_i$.

Zero Temperature Limit

Softmax:

$$\lim_{T \rightarrow 0} \text{softmax}(z/T)_i = \lim_{T \rightarrow 0} T \rightarrow 0 \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Substitute into the formula, then put the numerator $e^{z_i/T}$ into the denominator which has sum in z of the exponentials. We get $z_j - z_i$ in the exponent:

$$= \lim_{T \rightarrow 0} \frac{1}{\sum_j e^{(z_j - z_i)/T}}$$

In the limit when $z_j - z_i < 0$, the j -th component in the sum is 0. So for the argmax z_i , all components except the k -th are 0, the k -th being one due to $e^0 = 1$.

Sparsemax: For p_i to be non-zero, from the ReLU function we know that:

$$\frac{z_i}{T} > \tau \quad (2)$$

Now we also know that some z_i must be greater than zero, and a probability component is at most 1, so:

$$\frac{z_i}{T} - \tau \leq 1 \quad (3)$$

Now, with the intention to move towards the desired solution, which could otherwise be obtained numerically/computationally, we take z_i in equation (2) to be different from z_k , and z_i in equation (3) to be z_k , the maximum component of vector z , and combining both equations:

$$\frac{z_k}{T} - 1 \leq \tau < \frac{z_i}{T}$$

So:

$$\frac{z_k}{T} - 1 < \frac{z_i}{T} \implies \frac{z_k - z_i}{T} < 1 \implies z_k - z_i < T$$

Since $z_k - z_i > 0$ and T goes to 0^+ , in the limit the condition doesn't hold, so (2) doesn't hold, by contradiction. Notice only the limit being 0^+ and not any other positive value guarantees that all z_i are zero, and also that every component being different is also necessary.

Relumax:

$$\frac{\text{relu}(\frac{z_i - z_k}{T} + b)}{\sum_j \text{relu}(\frac{z_j - z_k}{T} + b)}$$

For $i \neq k$: $\frac{z_i - z_k}{T}$ tends to $-\infty$, so the numerator goes to zero. The denominator tends to b because of the term in the sum which gives $\frac{z_k - z_k}{T} + b = b$. $i = k$ makes the numerator b , which divided by b yields 1 and we get the desired result.

Relumax = Sparsemax Condition

For $\text{relumax}_b(z)_i$ to be equal to $\text{sparsemax}(z)_i$ we simply take the denominator of the latter (the normalization) to be one, and equal the arguments of the two ReLUs, obtaining:

$$z_i - \tau = z_i - z_k + b \iff b = z_k - \tau$$

And the denominator of $\text{relumax}_b(z)_i$ becomes $\sum_j (z_j - \tau)$ which is 1 due to sparsemax 's definition.

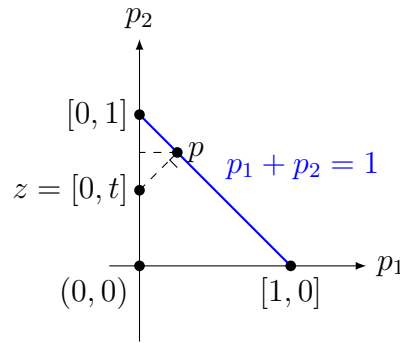
3.3

1. Softmax

$$P_2 = \text{softmax}(z)_2 = \frac{e^t}{e^0 + e^t} = \frac{e^t}{1 + e^t} = \sigma(t)$$

Derivative: $\sigma(t)(1 - \sigma(t))$.

2. Sparsemax



$p_1 + p_2 = 1$ is the line of the constraint, we just need to project $z = [0, t]$ orthogonally to the constraint, to minimize distance.

- If $t \geq 1$, obviously $\text{sparsemax}(z) = [0, 1]$.
- If $t \leq -1$, $\text{sparsemax}(z) = [1, 0]$.
- If $|t| < 1$:

$$\text{sparsemax}(z) = \left[\frac{1-t}{2}, \frac{1+t}{2} \right]$$

$p_2 = \frac{t+1}{2}$. Visually, it's the average between t and 1, as can be seen by the symmetry in the graph, and p_1 is just $1 - p_2$.

The derivative is 0.5 when $|t| < 1$ and 0 otherwise.

3. Relumax (Case $K = 2$)

$$P_2 = \frac{\text{relu}(t - \max(0, t) + b)}{\text{relu}(0 - \max(0, t) + b) + \text{relu}(t - \max(0, t) + b)}$$

If $t > 0$: $\max(0, t) = t$.

$$P_2 = \frac{\text{relu}(b)}{\text{relu}(b - t) + \text{relu}(b)} = \frac{b}{\text{relu}(b - t) + b}$$

If $t < b$:

$$P_2 = \frac{b}{(b - t) + b} = \frac{b}{2b - t}$$

If $t \leq 0$: $\max(0, t) = 0$.

$$P_2 = \frac{\text{relu}(t + b)}{\text{relu}(b) + \text{relu}(t + b)}$$

If $t > -b$:

$$P_2 = \frac{t + b}{b + t + b} = \frac{t + b}{2b + t}$$

3.4

We define $\text{relumax}(z)_i = \frac{n_i}{S}$, where $S = \sum_i n_i$. We say that n_i belongs to the set A if $z_i - z_k + b > 0$, and so $n_i > 0$. We will have a quotient rule for the Jacobian entry J_{ij} :

$$J_{ij} = \frac{\partial \text{relumax}(z)_i}{\partial z_j} = \frac{S \frac{\partial n_i}{\partial z_j} - n_i \frac{\partial S}{\partial z_j}}{S^2} = \frac{1}{S} \left(\frac{\partial n_i}{\partial z_j} - \text{relumax}(z)_i \frac{\partial S}{\partial z_j} \right)$$

If $i \notin A$, then $y_i = 0$. Also $n_i = 0$, so $\frac{\partial n_i}{\partial z_j} = 0$, and thus $J_{ij} = 0$.

Now considering the cases where $i \in A$:

- **Case 1:** $i \neq k$

Here $\frac{\partial n_i}{\partial z_j} = \delta_{ij} - \delta_{kj}$ and $\frac{\partial S}{\partial z_j} = 1$ (assuming $j \in A, j \neq k$).

– If $i = j$: Then $\frac{\partial n_i}{\partial z_i} = 1$ and $\frac{\partial S}{\partial z_i} = 1$.

$$J_{ii} = \frac{1}{S} (1 - \text{relumax}(z)_i)$$

– If $i \neq j$ and $j \neq k$: Then $\frac{\partial n_i}{\partial z_j} = 0$ and $\frac{\partial S}{\partial z_j} = 1$.

$$J_{ij} = -\frac{\text{relumax}(z)_i}{S}$$

– If $j = k$: Then $\frac{\partial n_i}{\partial z_k} = -1$ and $\frac{\partial S}{\partial z_k} = 1 - |A|$.

$$J_{ik} = \frac{1}{S} (-1 - \text{relumax}(z)_i (1 - |A|))$$

- **Case 2:** $i = k$

Here $\frac{\partial n_i}{\partial z_j} = 0$ because $n_k = b$ (constant).

- If $j \neq k$: Then $\frac{\partial S}{\partial z_j} = 1$.

$$J_{kj} = -\frac{\text{relumax}(z)_k}{S}$$

- If $j = k$: Then $\frac{\partial S}{\partial z_k} = 1 - |A|$.

$$J_{kk} = -\frac{\text{relumax}(z)_k}{S}(1 - |A|)$$

3.5

Both sparsemax and relumax, as seen by the plots, produce vanishing gradients, leading to numerical explosion, and are “less” smooth and differentiable than softmax, which can cause the same kinds of numerical explosions.

We consider the loss function:

$$L = -\log(p_i + \epsilon) + \log(c)$$

The gradient in z of L is given by the chain rule:

$$\frac{\partial L}{\partial z} = \frac{-1}{p_i + \epsilon} \nabla_z p_i$$

where $p_i = \text{relumax}(z)_i$ and thus $\nabla_z p_i$ is the i -th row of the Jacobian derived in the previous section. For the component z_j , we have:

$$\frac{\partial L}{\partial z_j} = \frac{-1}{p_i + \epsilon} J_{ij}$$

Bibliography

- [1] Gregory Cohen et al. “EMNIST: Extending MNIST to handwritten letters”. In: *2017 international joint conference on neural networks (IJCNN)*. IEEE. 2017, pp. 2921–2926.
- [2] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [3] Navneet Dalal and Bill Triggs. “Histograms of oriented gradients for human detection”. In: *CVPR*. 2005, pp. 886–893.

Use of Artificial Intelligence Tools

This appendix documents the use of Artificial Intelligence (AI) in the development of this project, what tools were used, and how they were used to help develop the project code and report.

Tools Used

- **Gemini Pro**
- **ChatGPT**
- **Claude**

Scope of Assistance

AI assistance was limited to the following areas:

- **Report preparation:** ChatGPT and Gemini Pro helped refine the text, clarify theoretical concepts, and assistance with LaTeX formatting issues.
- **Question 1:** Claude was used to diagnose and fix coding errors, as well as refactor code.
- **Question 2:** Gemini Pro assisted in understanding the `torch.nn.Module` class to correctly implement skeleton functions and methods, fix coding errors, and refactor code. Special impact on how to implement used libraries like `os` and `pandas`, and used for general suggestions on how to organize and segment the code.
- **Question 3:** chatGPT was used to quickly remember the definitions of functions like softmax and relu, hot vector. Gemini pro was used to understand the role of tau in 3.2, guidance with different cases in 3.4, and confirming ideas in 3.5. Gemini pro solutions were compared with the hand made solutions to compare and see missed details.



Verification and Human Oversight

All AI-generated content was manually reviewed and edited by the authors to ensure the code is working as intended according to the assignment and that no part of the project was solely or predominantly produced by AI. The authors maintained control over the entire project and made all significant contributions and decisions for the development of the project